

REACT ROADMAP

- **Requirements to set a react project.**

- Install - Node & NPM
- If already installed upgrade to the latest version
- Choose an editor like Visual Studio Code, IntelliJ, or your preferred choice of your editor.

- **What is React**

React is a JavaScript library for building user interfaces. It is declarative, efficient, and flexible.

React makes it easy to create interactive UIs by using a component-based approach.

Components are the building blocks of React applications. A component is a self-contained piece of UI that can be reused throughout an application. Components can be simple, like a button or a text input, or complex, like a navigation bar or a data table.

- **How will it work**

React operates by utilizing virtual DOM, which acts as a copy of the real DOM (Document Object Model) and reflects the layout and content of a webpage. By leveraging the DOM, React efficiently updates the altered components of the user interface without requiring complete page re-rendering.

```
const root = ReactDOM.createRoot(document.getElementById('root'));
root.render(<Car color="red"/>);
```

In the above code, It fetches a root element(div) in the HTML file and appends the component in the root element(div) without re-rendering the page.

- **React CLI tools**

React CLI tools are command-line tools that help developers create and manage React applications. They provide a variety of features, such as:

- ❖ Creating new React projects
- ❖ Managing React projects
- ❖ Debugging React applications

- **What is a component?**

Components are the building blocks of React applications. They are reusable pieces of code that can be used to create complex UIs. Components can be simple, like a button or a text input, or more complex, like a navigation bar or a modal dialog.

- **Components in JSX or TSX**

- ❖ **JSX - JavaScript XML**

JSX is a syntax extension for JavaScript that lets you write HTML-like markup inside a JavaScript file. Although there are other ways to write components, most React developers prefer the conciseness of JSX, and most codebases use it.

```
const myElement = (  
  <>  
    <p>I am a paragraph.</p>  
    <p>I am a paragraph too.</p>  
  </>  
);
```

- ❖ **TSX - TypeScript XML**

TSX extends JSX to be compatible with TypeScript, a statically typed superset of JavaScript. TypeScript adds optional static typing to JavaScript, providing features like type inference, interfaces, and type annotations. TSX allows you to write JSX syntax within TypeScript files (*.tsx) and provides type checking and autocompletion for JSX expressions.

Example:

```
function MyButton({ title }: { title: string }) {  
  return (  
    <button>{title}</button>  
  );  
}  
export default function MyApp() {  
  return (  
    <div>  
      <h1>Welcome to my app</h1>  
      <MyButton title="I'm a button" />  
    </div>  
  );  
}
```

Welcome to my app

I'm a button

- **How component works**

Components are independent and reusable bits of code. They serve the same purpose as JavaScript functions, but work in isolation and return HTML. Components come in two types, Class components and Function components.

- **Different types of components- Functional & Class**

- ❖ **Functional Component:**

A Function component also returns HTML and behaves much the same way as a Class component, but Function components can be written using much less code, are easier to understand, and will be preferred.

```
function Car() {  
  return <h2>Hi, I am a Car!</h2>;  
}
```

- ❖ **Class Component:**

A class component must include the extends React.Component statement. This statement creates an inheritance to React.Component, and gives your component access to React.Component's functions.

The component also requires a render() method, this method returns HTML.

```
class Car extends React.Component {  
  render() {  
    return <h2>Hi, I am a Car!</h2>;  
  }  
}
```

- **Component Basics**

- **props**

In React, "props" is short for "properties," and it's a way of passing data from a parent component to a child component. Props are immutable and are used to configure and customize child components.

Example:

```
// ParentComponent.js
import React from 'react';
import ChildComponent from './ChildComponent';

function ParentComponent() {
  // Define some data
  const data = {
    name: 'John',
    age: 30
  };

  return (
    <div>
      {/* Pass data to the ChildComponent */}
      <ChildComponent data={data} />
    </div>
  );
}

export default ParentComponent;
```

```
// ChildComponent.js
import React from 'react';

function ChildComponent(props) {
  // Access data passed from the parent component
  const { name, age } = props.data;

  return (
    <div>
      <p>Name: {name}</p>
      <p>Age: {age}</p>
    </div>
  );
}

export default ChildComponent;
```

- **State**

State is a built-in feature of class components and a feature that can be used in functional components with the introduction of React hooks. The state represents the current condition or data of the component, which can change over time due to user actions, network responses, or other events. When the state of a component changes, React re-renders the component to reflect those changes.

In class components, the state is declared in the constructor and can be updated using the `setState()` method, like so:

```
import React, { Component } from 'react';

// Define the type for the component's state
interface State {
  count: number;
}

class Counter extends Component<{}, State> {
  // Initialize the state
  state: State = {
    count: 0
  };

  // Function to handle the click event and update the count
  incrementCount = () => {
    this.setState({ count: this.state.count + 1 });
  };

  render() {
    return (
      <div>
        <p>Count: {this.state.count}</p>
        <button onClick={this.incrementCount}>Increment</button>
      </div>
    );
  }
}

export default Counter;
```

Similarly, In Functional Component

```
import React, { useState } from 'react';

function Counter() {
  // Define the state variable and a function to update it
  const [count, setCount] = useState<number>(0);

  // Function to handle the click event and update the count
  const incrementCount = () => {
    setCount(count + 1);
  };

  return (
    <div>
      <p>Count: {count}</p>
      <button onClick={incrementCount}>Increment</button>
    </div>
  );
}

export default Counter;
```

- **Context**

In React, "context" provides a way to pass data through the component tree without having to pass props down manually at every level. It's useful when some data needs to be accessible by many components at different nesting levels.

Context consists of two parts: the "provider" and the "consumer."

1. **Provider**: The provider component allows consuming components to subscribe to context changes. It accepts a value prop to be passed to the consuming components.
2. **Consumer**: The consumer component allows consuming the context value. It's used within the components that need to access the context data.

Example:

```
import React, { createContext, useContext } from 'react';

// Define the type for the context data
interface SharedState {
  message: string;
}
```

```

// Create a context
const MyContext = createContext<SharedState>({});

// Provider component
function MyProvider({ children }: { children: React.ReactNode }) {
  // Define the shared state
  const sharedState: SharedState = {
    message: 'Hello from Context!'
  };

  return <MyContext.Provider
value={sharedState}>{children}</MyContext.Provider>
}

// Consumer component
function ChildComponent() {
  // Use useContext hook to consume context
  const context = useContext(MyContext);
  if (!context) {
    throw new Error('useMyContext must be used within a MyProvider');
  }
  return <p>{context.message}</p>;
}

// Parent component
function ParentComponent() {
  return (
    <MyProvider>
      <ChildComponent />
    </MyProvider>
  );
}

export default ParentComponent;

```

- **Conditional Rendering**

Conditional rendering in React is the process of rendering different components or elements based on certain conditions.

Example:

```
import React from 'react';
```

```
interface Props {
  isLoggedIn: boolean;
}

const WelcomeMessage: React.FC<Props> = ({ isLoggedIn }) => {
  if (isLoggedIn) {
    return <h1>Welcome, User!</h1>;
  } else {
    return <h1>Please log in.</h1>;
  }
};

export default WelcomeMessage;
```

- **Composition**

Composition in React refers to the practice of combining multiple smaller components to build more complex and reusable components. It's a fundamental concept in React that promotes code reuse, maintainability, and readability.

There are two main ways to achieve composition in React:

1. **Component Composition:** This involves composing components together by nesting them within each other. Parent components can include child components by rendering them within their JSX markup. Each component encapsulates a specific piece of functionality or UI, and they can be combined to create larger, more complex UIs.

Example:

```
import React from 'react';

// Header component
interface HeaderProps {
  title: string;
}

const Header: React.FC<HeaderProps> = ({ title }) => {
  return <h1>{title}</h1>;
};

// Content component
interface ContentProps {
  text: string;
```



```

}

const Content: React.FC<ContentProps> = ({ text }) => {
  return <p>{text}</p>;
};

// Page component composed of Header and Content
interface PageProps {
  title: string;
  content: string;
}

const Page: React.FC<PageProps> = ({ title, content }) => {
  return (
    <div>
      <Header title={title} />
      <Content text={content} />
    </div>
  );
};

// App component rendering Page
const App: React.FC = () => {
  return (
    <div>
      <Page title="Welcome to my website" content="This is the content of
the page." />
    </div>
  );
};

export default App;

```

2. **Higher-order components (HOCs):** Higher-order components are functions that take a component as an input and return a new enhanced component with additional functionality. They enable code reuse by separating concerns and applying the same logic to multiple components.

- **Rendering**

- **Component life cycle**

In React, class components have a lifecycle that consists of several phases or stages during which the component is created, rendered, updated, and destroyed. These phases are called "lifecycle methods" or "component lifecycle hooks". With the introduction of

React Hooks, functional components can also have lifecycle-like behavior using the `useEffect` hook.

Here's a summary of the lifecycle methods/hooks for class components:

1. Mounting Phase:
 - `constructor()`: Initialize state and bind event handlers.
 - `componentDidMount()`: Perform setup after the component is mounted.
2. Updating Phase:
 - `shouldComponentUpdate(nextProps, nextState)`: Decide if the component should re-render.
 - `componentDidUpdate(prevProps, prevState)`: Perform side effects after an update.
3. Unmounting Phase:
 - `componentWillUnmount()`: Clean up before the component is removed.
4. Error Handling:
 - `componentDidCatch(error, info)`: Handle errors that occur during rendering.

Example:

```
import React, { Component } from 'react';

interface State {
  count: number;
}

class ExampleComponent extends Component<{}, State> {
  private timer: NodeJS.Timeout | null = null;

  constructor(props: {}) {
    super(props);
    this.state = {
      count: 0
    };
    console.log('Constructor executed');
  }

  componentDidMount() {
    console.log('Component did mount');
    // Simulating an asynchronous operation
    this.timer = setInterval(() => {
      this.setState({ count: this.state.count + 1 });
    }, 1000);
  }
}
```

```

componentDidUpdate(prevProps: {}, prevState: State) {
  console.log('Component did update');
}

componentWillUnmount() {
  console.log('Component will unmount');
  if (this.timer) {
    clearInterval(this.timer); // Cleanup timer
  }
}

render() {
  console.log('Render');
  return <div>Count: {this.state.count}</div>;
}
}

export default ExampleComponent;

```

In functional components, you can use the `useEffect` hook to achieve similar behaviors.

- **List & keys**

In React, when rendering a list of elements, it's common to use the `map()` function to iterate over an array and generate a list of React elements. When doing so, React requires that each item in the list has a unique `key` prop assigned to it. This helps React efficiently identify which items have changed, been added, or been removed, improving performance and ensuring proper rendering.

Example:

```

import React from 'react';

interface ListItem {
  id: number;
  text: string;
}

interface ListProps {
  items: ListItem[];
}

const MyListComponent: React.FC<ListProps> = ({ items }) => {
  return (
    <ul>

```

```

    {items.map(item => (
      <li key={item.id}>{item.text}</li>
    ))}
  </ul>
);
};

export default MyListComponent;

```

- **Render props**

Render props is a pattern in React where a component's functionality is provided through a prop that is a function. This allows components to share code and behavior with other components by letting the user of the component decide what to render. Here's how it works, along with an example in TypeScript (.tsx):

```

import React, { useState } from 'react';

interface CounterProps {
  children: (count: number, increment: () => void) => React.ReactNode;
}

const Counter: React.FC<CounterProps> = ({ children }) => {
  const [count, setCount] = useState(0);

  const increment = () => {
    setCount(prevCount => prevCount + 1);
  };

  return <>{children(count, increment)}</>;
};

export default Counter;

```

```

import React from 'react';
import Counter from './Counter';

const ParentComponent: React.FC = () => {
  return (
    <Counter>
      {(count, increment) => (
        <div>
          <p>Count: {count}</p>

```

```

        <button onClick={increment}>Increment</button>
      </div>
    )}
  </Counter>
);
};

export default ParentComponent;

```

- **Refs**

In React, refs are a way to access and interact with DOM elements or React components directly. They provide a way to reference a particular element or component in your application. Refs are commonly used for the following purposes:

1. Accessing DOM Elements
2. Managing Focus, Selection, and Media Playback
3. Integrating with Third-Party Libraries
4. Communicating Between Parent and Child Components

Example:

```

import React, { useRef } from 'react';

const MyComponent: React.FC = () => {
  const inputRef = useRef<HTMLInputElement>(null);

  const handleClick = () => {
    // Focus the input element
    if (inputRef.current) {
      inputRef.current.focus();
    }
  };

  return (
    <div>
      <input ref={inputRef} type="text" />
      <button onClick={handleClick}>Focus Input</button>
    </div>
  );
};

export default MyComponent;

```

- **Events**

In React, events are interactions that occur in the user interface, such as clicking a button, hovering over an element, typing into an input field, etc. React provides a synthetic event system that wraps the native browser events and provides a consistent interface across different browsers.

1. **Event Handling:** In React, you can attach event handlers to React elements using JSX syntax. You typically define event handlers as functions and pass them as props to the elements you want to listen for events on.

```
import React from 'react';

const handleClick = () => {
  console.log('Button clicked!');
};

const MyButton: React.FC = () => {
  return <button onClick={handleClick}>Click me</button>;
};

export default MyButton;
```

2. **Event Objects:** When an event occurs, React automatically passes a synthetic event object to the event handler function. This object contains information about the event, such as the type of event, the target element, and any additional event-specific data.

```
import React from 'react';

const handleInputChange = (event: React.ChangeEvent<HTMLInputElement>) => {
  console.log('Input value:', event.target.value);
};

const MyInput: React.FC = () => {
  return <input type="text" onChange={handleInputChange} />;
};

export default MyInput;
```

3. **Event Binding:** Inside event handler functions, you can access the event object and perform any necessary logic, such as updating the state, invoking other functions, or modifying the DOM. React automatically binds the event handler function to the component instance, so you don't need to worry about losing the context of this.

```
import React, { useState } from 'react';

const MyButton: React.FC = () => {
  const [clicked, setClicked] = useState(false);

  const handleClick = () => {
    setClicked(true);
  };

  return (
    <button onClick={handleClick}>
      {clicked ? 'Button Clicked!' : 'Click me'}
    </button>
  );
};

export default MyButton;
```

4. **Event Propagation:** React follows the same event propagation model as the native DOM. Events bubble up from the target element to its ancestors, allowing you to capture events at different levels of the component hierarchy.

```
import React from 'react';

const handleChildClick = () => {
  console.log('Child clicked!');
};

const ChildComponent: React.FC = () => {
  return <button onClick={handleChildClick}>Click me</button>;
};

const ParentComponent: React.FC = () => {
  const handleParentClick = () => {
    console.log('Parent clicked!');
  };

  return (
    <div onClick={handleParentClick}>
      <ChildComponent />
    </div>
  );
};
```

```
};

export default ParentComponent;
```

5. **Event Pooling:** React reuses event objects for performance reasons. After an event handler function is called, the event object is reset and returned to the event pool, so you can't access event properties asynchronously. If you need to access event properties asynchronously, you can call `event.persist()` to remove the event from the pool.

```
import React from 'react';

const handleInputChange = (event: React.ChangeEvent<HTMLInputElement>) => {
  // Asynchronously logging the event object
  setTimeout(() => {
    console.log('Input value:', event.target.value);
  }, 0);
};

const MyInput: React.FC = () => {
  return <input type="text" onChange={handleInputChange} />;
};

export default MyInput;
```

- **Error boundaries**

Error boundaries in React are special components that catch JavaScript errors anywhere in their child component tree, log those errors, and display a fallback UI instead of crashing the entire application. They provide a way to gracefully handle errors that occur during rendering, in lifecycle methods, or the constructors of any component in their subtree.

```
import React, { useState } from 'react';

interface ErrorBoundaryProps {
  children: React.ReactNode;
}

interface ErrorBoundaryState {
  hasError: boolean;
}

const ErrorBoundary: React.FC<ErrorBoundaryProps> = ({ children }) => {
```



```

const [errorState, setErrorState] = useState<ErrorBoundaryState>({
  hasError: false });

const componentDidCatch = (error: Error, errorInfo: React.ErrorInfo) => {
  // Log the error to an error reporting service
  console.error('Error caught by error boundary:', error, errorInfo);
  // Update state to indicate that an error occurred
  setErrorState({ hasError: true });
};

if (errorState.hasError) {
  // Render fallback UI if an error occurred
  return <h1>Something went wrong.</h1>;
}

// Render children normally
return <>{children}</>;
};

export default ErrorBoundary;

```

- **High order components**

Higher-order components are functions that take a component as an input and return a new enhanced component with additional functionality. They enable code reuse by separating concerns and allowing the same logic to be applied to multiple components.

```

import React, { ComponentType } from 'react';

// Define a Higher-Order Component
function withUpperCase<P>(WrappedComponent: ComponentType<P>) {
  // Return a new component
  return (props: P) => {
    // Modify props before passing them to the wrapped component
    const modifiedProps = { ...props, text: (props.text as
string).toUpperCase() };
    return <WrappedComponent {...modifiedProps} />;
  };
}

// Example component
const MyComponent: React.FC<{ text: string }> = ({ text }) => {
  return <div>{text}</div>;
}

```

```

};

// Enhance MyComponent with the HOC
const MyComponentWithUpperCase = withUpperCase(MyComponent);

// Usage
const App: React.FC = () => {
  return <MyComponentWithUpperCase text="hello, world!" />;
};

export default App;

```

- **Hooks**

Hooks is a feature introduced in React 16.8 that allows you to use state and other React features in functional components without writing a class. They enable you to reuse stateful logic between components, making it easier to manage stateful logic in functional components.

- **Basic hooks - use state & use effect**

- **useState**

useState is a hook that allows functional components to use state. It returns a stateful value and a function to update it. You can call useState multiple times in a single component to manage multiple state variables.

```

import React, { useState } from 'react';

const Counter: React.FC = () => {
  // useState hook to manage state
  const [count, setCount] = useState<number>(0);

  return (
    <div>
      <p>Count: {count}</p>
      <button onClick={() => setCount(count + 1)}>Increment</button>
      <button onClick={() => setCount(count - 1)}>Decrement</button>
    </div>
  );
};

export default Counter;

```

UseEffect

useEffect is a hook that allows functional components to perform side effects. It's similar to lifecycle methods in class components (like componentDidMount, componentDidUpdate, and componentWillUnmount). You can use it to fetch data, subscribe to events, or perform cleanup.

```
import React, { useState, useEffect } from 'react';

const Timer: React.FC = () => {
  // useState hook to manage state
  const [seconds, setSeconds] = useState<number>(0);

  // useEffect hook to perform side effects
  useEffect(() => {
    // Function to increment seconds every second
    const intervalId = setInterval(() => {
      setSeconds((prevSeconds) => prevSeconds + 1);
    }, 1000);

    // Cleanup function to clear the interval when the component unmounts
    return () => {
      clearInterval(intervalId);
    };
  }, []); // Empty dependency array ensures effect runs only once after
  initial render

  return <p>Seconds elapsed: {seconds}</p>;
};

export default Timer;
```

- Common hooks - <https://react.dev/reference/react/hooks> - Optional

- **useCallback**

useCallback is a hook in React that returns a memoized callback function. It is used to optimize performance by memoizing functions and preventing unnecessary re-renders in components that rely on callbacks.

```
import React, { useState, useCallback } from 'react';

const Counter: React.FC = () => {
  const [count, setCount] = useState<number>(0);

  // Define a callback function using useCallback
  const increment = useCallback(() => {
```

```

    setCount(prevCount => prevCount + 1);
  }, []);

  // Define a callback function using useCallback with dependencies
  const decrement = useCallback(() => {
    setCount(prevCount => prevCount - 1);
  }, []);

  return (
    <div>
      <p>Count: {count}</p>
      /* Pass the memoized callbacks to the onClick event handlers */
      <button onClick={increment}>Increment</button>
      <button onClick={decrement}>Decrement</button>
    </div>
  );
};
export default Counter;

```

■ [useContext](#)

useContext is a React hook that allows functional components to consume context. It provides a way for components to access values from a context without having to pass props through every level of the component tree.

```

import React, { createContext, useContext } from 'react';

// Step 1: Create a context
const ThemeContext = createContext<string>('light');

// Step 2: Provide context value
const App: React.FC = () => {
  return (
    <ThemeContext.Provider value="dark">
      <Toolbar />
    </ThemeContext.Provider>
  );
};

```

```
// Step 3: Consume context value
const Toolbar: React.FC = () => {
  const theme = useContext(ThemeContext);
  return <div>Current theme: {theme}</div>;
};

export default App;
```

■ [useDebugValue](#)

useDebugValue is a React hook used for debugging purposes. It provides a way to display a custom label for custom hooks in React DevTools, making it easier to identify custom hooks during debugging.

```
import { useDebugValue, useState } from 'react';

function useCustomHook(initialValue: string) {
  const [value, setValue] = useState(initialValue);

  // Display the value in React DevTools with a custom label
  useDebugValue(value, (value) => `Custom Hook Value: ${value}`);

  return { value, setValue };
}
```

■ [useReducer](#)

useReducer is a React hook that is used for state management, similar to useState, but more suitable for managing complex state logic that involves multiple sub-values or when the next state depends on the previous one.

```
import React, { useReducer } from 'react';

// Define the type for state
interface State {
  count: number;
}
```

```

// Define the type for action
type Action = { type: 'INCREMENT' } | { type: 'DECREMENT' };

// Reducer function
const reducer = (state: State, action: Action): State => {
  switch (action.type) {
    case 'INCREMENT':
      return { count: state.count + 1 };
    case 'DECREMENT':
      return { count: state.count - 1 };
    default:
      return state;
  }
};

const Counter: React.FC = () => {
  // useReducer hook takes a reducer function and an initial state
  const [state, dispatch] = useReducer(reducer, { count: 0 });

  return (
    <div>
      <p>Count: {state.count}</p>
      <button onClick={() => dispatch({ type: 'INCREMENT'
    })}>Increment</button>
      <button onClick={() => dispatch({ type: 'DECREMENT'
    })}>Decrement</button>
    </div>
  );
};

export default Counter;

```

■ [useRef](#)

useRef is a React hook that provides a way to create a mutable ref object that persists across renders. It is commonly used to access or store references to DOM elements or other mutable values.

```
import React, { useRef, useEffect } from 'react';

const MyComponent = () => {
  // Create a ref object
  const inputRef = useRef<HTMLInputElement>(null);

  // Use the ref to focus the input element on mount
  useEffect(() => {
    if (inputRef.current) {
      inputRef.current.focus();
    }
  }, []);

  return (
    <div>
      { /* Attach the ref to the input element */ }
      <input ref={inputRef} type="text" />
    </div>
  );
};

export default MyComponent;
```

- **How to create custom hooks**

Creating custom hooks in React is a way to encapsulate and share logic between different components. Custom hooks are regular JavaScript functions that use built-in hooks or other custom hooks to manage state, side effects, or any other logic.

```
import { useState, useEffect } from 'react';

// Define the type for data
interface Data {
  // Define the structure of your data
}

// Custom hook to fetch data from an API
const useFetchData = (url: string) => {
  const [data, setData] = useState<Data | null>(null);
  const [isLoading, setIsLoading] = useState<boolean>(true);
  const [error, setError] = useState<Error | null>(null);

  useEffect(() => {
```

```

const fetchData = async () => {
  try {
    const response = await fetch(url);
    if (!response.ok) {
      throw new Error('Failed to fetch data');
    }
    const result: Data = await response.json();
    setData(result);
    setIsLoading(false);
  } catch (error) {
    setError(error);
    setIsLoading(false);
  }
};

fetchData();

// Cleanup function
return () => {
  // Cleanup logic, if any
};
}, [url]); // Dependency array to re-run effect when URL changes

return { data, isLoading, error };
};

export default useFetchData;

```

- **Virtual DOM**

The Virtual DOM (Document Object Model) is a concept in React that represents an abstract, in-memory representation of the actual DOM. It is a lightweight copy of the real DOM and is used by React to improve performance and efficiency in updating the UI.

Here's how the Virtual DOM works in React:

1. **Initial Render:** When you create a React component and render it for the first time, React constructs a virtual representation of the UI called the Virtual DOM. This Virtual DOM is a JavaScript object that mirrors the structure of the actual DOM but contains only the necessary information to render the UI.

2. **Updates:** Whenever there is a change in the component's state or props, React re-renders the component and generates a new Virtual DOM representation.
3. **Diffing:** React then compares the new Virtual DOM with the previous Virtual DOM to identify the differences (or "diffs") between them. This process is known as reconciliation.
4. **Minimal Updates:** After identifying the differences, React determines the minimal set of changes needed to update the actual DOM to match the new Virtual DOM. Instead of updating the entire DOM tree, React only applies the specific changes required to reflect the updated UI.
5. **Batching:** React batches multiple updates together to minimize the number of DOM manipulations. This helps improve performance by reducing the frequency of re-renders and DOM updates.
6. **Updating the DOM:** Finally, React applies the calculated changes to the actual DOM, updating only the elements that have changed.

- **Forms**

In React, handling forms involves managing form data, handling form submission, and updating the UI based on user input. React provides several approaches to handle forms, including controlled components and uncontrolled components.

1. **Controlled Components:** In controlled components, form elements like `<input>`, `<textarea>`, and `<select>` are controlled by React state. You keep the form data in React state and update it in response to user input, which allows you to control and validate the input easily.

```
import React, { useState } from 'react';

const MyForm = () => {
  const [formData, setFormData] = useState({
    username: '',
    password: ''
  });

  const handleChange = (event) => {
    const { name, value } = event.target;
    setFormData({ ...formData, [name]: value });
  };

  const handleSubmit = (event) => {
    event.preventDefault();
    // Handle form submission using formData
  };
}
```

```

    };
    return (
      <form onSubmit={handleSubmit}>
        <input type="text" name="username" value={formData.username}
onChange={handleChange} />
        <input type="password" name="password" value={formData.password}
onChange={handleChange} />
        <button type="submit">Submit</button>
      </form>
    );
  };
}
export default MyForm;

```

2. **Uncontrolled Components:** In uncontrolled components, form elements maintain their state and rely on refs to access their values. Uncontrolled components can be useful when integrating with non-React libraries or when you want to minimize React's involvement in managing form state.

```

import React, { useRef } from 'react';

const MyForm = () => {
  const usernameRef = useRef(null);
  const passwordRef = useRef(null);

  const handleSubmit = (event) => {
    event.preventDefault();
    const username = usernameRef.current.value;
    const password = passwordRef.current.value;
    // Handle form submission using username and password
  };

  return (
    <form onSubmit={handleSubmit}>
      <input type="text" ref={usernameRef} />
      <input type="password" ref={passwordRef} />
      <button type="submit">Submit</button>
    </form>
  );
};

export default MyForm;

```

- **Routers**

In React, routers are used for navigation and managing the URLs of your application. They allow you to define different routes in your application and render different components based on the current URL.

- **React router**

React Router is the most popular routing library for React applications. It provides a declarative way to define routes and navigate between them.

```
// App.tsx
import React from 'react';
import { BrowserRouter as Router, Route, Routes } from 'react-router-dom';
import Home from './Home';
import About from './About';
import Contact from './Contact';
import NotFound from './NotFound';

const App: React.FC = () => {
  return (
    <Router>
      <Routes>
        <Route path="/" exact component={Home} />
        <Route path="/about" component={About} />
        <Route path="/contact" component={Contact} />
        <Route component={NotFound} />
      </Routes>
    </Router>
  );
};

export default App;
```

- **Reach router - a subset of react-router**

React Router v6, often referred to as Reach Router, is a routing library for React applications. It's a lightweight, accessible, and easy-to-use library for managing routes and navigation in your React apps.

Reach Router is designed to be more accessible and to support declarative routing with a focus on accessibility features such as screen reader support and keyboard navigation.

```
// App.tsx
import React from 'react';
import { Router, Route, Routes } from '@reach/router';
```

```

import Home from './Home';
import About from './About';
import Contact from './Contact';
import NotFound from './NotFound';

const App: React.FC = () => {
  return (
    <Router>
      <Routes>
        <Route path="/" element={<Home />} />
        <Route path="/about" element={<About />} />
        <Route path="/contact" element={<Contact />} />
        <Route path="*" element={<NotFound />} />
      </Routes>
    </Router>
  );
};

export default App;

```

- **State management**

State management in React typically involves managing the state of your application's components and sharing state between components that are not directly related in the component tree.

- **Redux/toolkit**

Redux is a predictable state container for JavaScript applications, including React. It helps manage the application state in a more organized and centralized way. Redux can be integrated into React applications to handle complex state management needs.

- **Store setup**

Create a Redux store that holds the global state of your application.

```

// store.ts
import { createStore } from 'redux';
import rootReducer from './reducers';

const store = createStore(rootReducer);

export default store;

```

Define reducers to specify how the application's state changes in response to actions.

```
// counterReducer.ts
interface CounterState {
  count: number;
}

const initialState: CounterState = {
  count: 0
};

const counterReducer = (state: CounterState = initialState, action: any): CounterState => {
  switch (action.type) {
    case 'INCREMENT':
      return { ...state, count: state.count + 1 };
    case 'DECREMENT':
      return { ...state, count: state.count - 1 };
    default:
      return state;
  }
};

export default counterReducer;
```

Combine all reducers into a single rootReducer.

```
// rootReducer.ts
import { combineReducers } from 'redux';
import counterReducer from './counterReducer';

const rootReducer = combineReducers({
  counter: counterReducer
});

export default rootReducer;
```

Provide Redux Store to React:

Wrap your root component with the Redux Provider and pass the Redux store as a prop.

```
// App.tsx
import React from 'react';
import { Provider } from 'react-redux';
import store from './store';
import MyComponent from './MyComponent';

const App: React.FC = () => {
  return (
    <Provider store={store}>
      <MyComponent />
    </Provider>
  );
};

export default App;
```

Connect Components to Redux:

Connect your components to the Redux store using the connect function from react-redux.

```
// MyComponent.tsx
import React from 'react';
import { connect } from 'react-redux';
import { RootState } from './reducers';

interface MyComponentProps {
  count: number;
  increment: () => void;
  decrement: () => void;
}

const MyComponent: React.FC<MyComponentProps> = ({ count, increment,
decrement }) => {
  return (
    <div>
      <p>Count: {count}</p>
      <button onClick={increment}>Increment</button>
      <button onClick={decrement}>Decrement</button>
    </div>
  );
};

const mapStateToProps = (state: RootState) => ({
```

```

    count: state.counter.count
  });

  const mapDispatchToProps = (dispatch: any) => ({
    increment: () => dispatch({ type: 'INCREMENT' }),
    decrement: () => dispatch({ type: 'DECREMENT' })
  });

  export default connect(mapStateToProps, mapDispatchToProps)(MyComponent);

```

■ Slice API

The Redux Toolkit provides a powerful utility called "Slice" that simplifies the process of defining reducers, action creators, and initial state for a specific slice of the Redux store. It encapsulates all the logic related to managing a specific piece of state, making your code more modular and easier to maintain.

Create a new file for your slice and use the createSlice function to define it. This function takes an object with three fields: name, initialState, and reducers.

```

// counterSlice.ts
import { createSlice, PayloadAction } from '@reduxjs/toolkit';

interface CounterState {
  value: number;
}

const initialState: CounterState = {
  value: 0,
};

const counterSlice = createSlice({
  name: 'counter',
  initialState,
  reducers: {
    increment(state) {
      state.value++;
    },
    decrement(state) {
      state.value--;
    },
    incrementByAmount(state, action: PayloadAction<number>) {
      state.value += action.payload;
    },
  },
});

```

```
});
export const { increment, decrement, incrementByAmount } =
  counterSlice.actions;
export default counterSlice.reducer;
```

Combine your slice reducer with other reducers using the `combineReducers` utility provided by Redux Toolkit.

```
// rootReducer.ts
import { combineReducers } from '@reduxjs/toolkit';
import counterReducer from './counterSlice';

const rootReducer = combineReducers({
  counter: counterReducer,
});

export type RootState = ReturnType<typeof rootReducer>;
export default rootReducer;
```

Create Store and Provide to React:

```
// store.ts
import { configureStore } from '@reduxjs/toolkit';
import rootReducer from './reducers';

const store = configureStore({
  reducer: rootReducer,
});

export default store;
```

Connect Components to Redux:

Connect your components to the Redux store using the `useDispatch` hook to dispatch actions and the `useSelector` hook to select state.

```
// MyComponent.tsx
import React from 'react';
import { useDispatch, useSelector } from 'react-redux';
import { RootState } from './reducers';
import { increment, decrement, incrementByAmount } from './counterSlice';

const MyComponent: React.FC = () => {
  const dispatch = useDispatch();
  const count = useSelector((state: RootState) => state.counter.value);

  return (
```



```

    <div>
      <p>Count: {count}</p>
      <button onClick={() => dispatch(increment())}>Increment</button>
      <button onClick={() => dispatch(decrement())}>Decrement</button>
      <button onClick={() => dispatch(incrementByAmount(5))}>Increment by
5</button>
    </div>
  );
};

export default MyComponent;

```

■ Immutable updates

In Redux, it's important to maintain immutability when updating the state. This means that you should not directly mutate the existing state, but instead create a new copy of the state with the desired changes. This ensures that Redux can properly detect state changes and trigger UI updates efficiently.

1. Redux Toolkit includes createReducer and createSlice utilities with Immer.
2. Immer allows writing code as if directly modifying state, but it handles immutability behind the scenes.

```

import { createSlice, PayloadAction } from '@reduxjs/toolkit';

interface CounterState {
  value: number;
}

const initialState: CounterState = {
  value: 0,
};

const counterSlice = createSlice({
  name: 'counter',
  initialState,
  reducers: {
    increment: (state: CounterState) => {
      state.value++; // Immer handles immutability
    },
  },
});

export const { increment } = counterSlice.actions;

```

```
export default counterSlice.reducer;
```

■ Actions

In React Redux, actions are plain JavaScript objects that represent payloads of information that send data from your application to the Redux store. They are the only source of information for the store. Actions must have a type property indicating the type of action being performed. They can also have additional data attached to them as the payload.

```
// actionTypes.tsx
import React from 'react';

export const INCREMENT: React.ReactNode = 'INCREMENT';
export const DECREMENT: React.ReactNode = 'DECREMENT';
export const ADD_TODO: React.ReactNode = 'ADD_TODO';
```

■ Reducers

In React Redux, reducers are functions that specify how the application's state changes in response to actions dispatched to the Redux store. Reducers take the current state and an action as arguments, and return the new state based on the action type.

```
// reducer.ts
import { initialState, AppState } from './initialState';
import { ActionTypes } from './actionTypes';

const reducer = (state: AppState = initialState, action: ActionTypes):
AppState => {
  switch (action.type) {
    case 'INCREMENT':
      return { ...state, count: state.count + 1 };
    case 'DECREMENT':
      return { ...state, count: state.count - 1 };
    case 'ADD_TODO':
      return { ...state, todos: [...state.todos, action.payload] };
    default:
      return state;
  }
};

export default reducer;
```

■ Selectors

In React Redux, selectors are functions that extract specific pieces of data from the Redux store state. They allow you to compute derived states or perform complex data transformations while keeping your components decoupled from the shape of the state.

```
// selectors.ts
import { RootState } from './reducers';

export const selectCount = (state: RootState) => state.counter.count;
```

In this example, selectCount is a selector function that takes the root state of the Redux store as an argument and returns the count property from the counter slice of the state.

```
// selectors.ts
import { RootState } from './reducers';
import { createSelector } from '@reduxjs/toolkit';

export const selectCompletedTodos = createSelector(
  (state: RootState) => state.todos,
  todos => todos.filter(todo => todo.completed)
);
```

In this example, selectCompletedTodos is a selector function created using the createSelector utility from Redux Toolkit. It takes the root state as an argument and returns an array of completed todos by filtering the todos from the state.

■ Error handling

Error handling in React Redux involves managing errors that occur during asynchronous operations, such as network requests or other side effects. Redux middleware, such as redux-thunk or redux-saga, is commonly used to handle asynchronous actions and error propagation.

Approaches:

1. Dispatching Asynchronous Actions:

When performing asynchronous operations, such as fetching data from an API, dispatch an action to indicate the start of the operation. If an error occurs during the operation, dispatch an action to handle the error.

```
// actions.ts
import { Dispatch } from 'redux';
import axios from 'axios';

export const fetchData = () => {
  return async (dispatch: Dispatch) => {
    dispatch({ type: 'FETCH_DATA_REQUEST' });
```

```

    try {
      const response = await axios.get('https://api.example.com/data');
      dispatch({ type: 'FETCH_DATA_SUCCESS', payload: response.data });
    } catch (error) {
      dispatch({ type: 'FETCH_DATA_FAILURE', payload: error.message });
    }
  };
};

```

2. Handling Errors in Reducers:

Handle the error in your reducers by updating the state to reflect the error condition. For example, you might set an error flag in the state and store the error message.

```

// reducers.ts
interface State {
  data: any;
  loading: boolean;
  error: string | null;
}

const initialState: State = {
  data: null,
  loading: false,
  error: null,
};

const reducer = (state = initialState, action: any) => {
  switch (action.type) {
    case 'FETCH_DATA_REQUEST':
      return { ...state, loading: true, error: null };
    case 'FETCH_DATA_SUCCESS':
      return { ...state, loading: false, data: action.payload };
    case 'FETCH_DATA_FAILURE':
      return { ...state, loading: false, error: action.payload };
    default:
      return state;
  }
};

export default reducer;

```

3. Displaying Errors in Components:

In your React components, conditionally render error messages based on the state stored in Redux.

```
// MyComponent.tsx
import React, { useEffect } from 'react';
import { useDispatch, useSelector } from 'react-redux';
import { RootState } from './reducers';
import { fetchData } from './actions';

const MyComponent: React.FC = () => {
  const dispatch = useDispatch();
  const { data, loading, error } = useSelector((state: RootState) => state);

  useEffect(() => {
    dispatch(fetchData());
  }, [dispatch]);

  if (loading) {
    return <div>Loading...</div>;
  }

  if (error) {
    return <div>Error: {error}</div>;
  }

  return (
    <div>
      { /* Render data */ }
    </div>
  );
};

export default MyComponent;
```

■ Middleware

Middleware in Redux intercepts actions before they reach the reducer, providing a way to implement cross-cutting concerns such as logging, asynchronous operations, or authentication checks. Middleware is an essential part of Redux architecture, enabling you to extend Redux's functionality in a composable way.

1. Applying Middleware:

You can apply middleware when creating the Redux store using the `applyMiddleware` function from Redux. Middleware functions are composed using this function, and they are applied in the order they are provided.

```
// store.ts
import { createStore, applyMiddleware } from 'redux';
import thunk from 'redux-thunk'; // Example middleware
import rootReducer from './reducers';

const store = createStore(
  rootReducer,
  applyMiddleware(thunk) // Apply middleware
);

export default store;
```

2. Creating Middleware:

Middleware is a function that takes the store object as its first argument and returns a function that takes the next middleware in the chain as its argument. It also receives the action being dispatched.

```
// loggerMiddleware.ts
const loggerMiddleware = (store: any) => (next: any) => (action: any) => {
  console.log('Dispatching action:', action);
  const result = next(action);
  console.log('Updated state:', store.getState());
  return result;
};

export default loggerMiddleware;
```

3. Applying Custom Middleware:

You can apply custom middleware alongside existing middleware when creating the Redux store.

```
// store.ts
import { createStore, applyMiddleware } from 'redux';
import thunk from 'redux-thunk'; // Example middleware
import rootReducer from './reducers';
import loggerMiddleware from './loggerMiddleware'; // Custom middleware

const store = createStore(
  rootReducer,
  applyMiddleware(thunk, loggerMiddleware) // Apply custom middleware
);

export default store;
```

■ Debugging & Testing

Debugging and testing are crucial aspects of developing React Redux applications to ensure the stability and reliability of your code. Here's how you can approach debugging and testing in React Redux:

- Debugging
- Unit Testing
- Integration Testing
- End-to-End Testing

● Styling

○ CSS - Cascading Style Sheet

In React, you have several options for styling your components. These options include traditional CSS files, CSS modules, inline styles, CSS-in-JS libraries, and CSS preprocessors like Sass or Less.

Example:

```
/* MyComponent.css */
.container {
  background-color: #f0f0f0;
  padding: 20px;
}

.title {
  color: blue;
  font-size: 24px;
```

```
}
```

- **SCSS**

SCSS (Sassy CSS) is a superset of CSS that offers additional features and functionality to make styling more efficient and maintainable.

- **Variables**

```
// styles.scss
$primary-color: blue;

.container {
  background-color: #f0f0f0;
  padding: 20px;
}

.title {
  color: $primary-color;
  font-size: 24px;
}
```

- **Nesting**

```
.container {
  background-color: #f0f0f0;
  padding: 20px;

  .title {
    color: $primary-color;
    font-size: 24px;
  }
}
```

- **Mixins**

```
@mixin flexbox {
  display: flex;
  justify-content: center;
  align-items: center;
}

.container {
  @include flexbox;
  background-color: #f0f0f0;
  padding: 20px;
}
```


➤ **Partials and Importing**

SCSS allows you to split your stylesheets into smaller files called partials and import them into other files

Example: `_variables.scss`

```
// _variables.scss
$primary-color: blue;
```

```
@import 'variables';

.container {
  background-color: $primary-color;
  padding: 20px;
}
```

➤ **Functions**

SCSS provides built-in functions for common tasks, such as color manipulation and math operations.

```
.container {
  background-color: darken($primary-color, 10%);
}
```

➤ **Extend / Inheritance**

SCSS allows you to extend styles from one selector to another, promoting code reusability.

```
.btn {
  padding: 10px;
  border: 1px solid black;
}

.primary-btn {
  @extend .btn;
  background-color: $primary-color;
  color: white;
}
```

- **API calls**

In React applications, you can make API calls to fetch data from a server using various methods.

- **Rest API**

A REST API (Representational State Transfer Application Programming Interface) is an architectural style for designing networked applications. It is based on a set of principles that outline how resources are defined and addressed over the web.

- **Fetch API**

Use the Async Await Method to fetch the data.

```
import React, { useState, useEffect } from 'react';

const MyComponent = () => {
  const [data, setData] = useState(null);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    const fetchData = async () => {
      try {
        const response = await fetch('https://api.example.com/data');
        const jsonData = await response.json();
        setData(jsonData);
        setLoading(false);
      } catch (error) {
        console.error('Error fetching data:', error);
      }
    };

    fetchData();
  }, []);

  if (loading) {
    return <div>Loading...</div>;
  }

  return (
    <div>
      {/* Display fetched data */}
      {data && (
        <ul>
          {data.map(item => (
            <li key={item.id}>{item.name}</li>
          ))}
        </ul>
      )}
    </div>
  );
}
```

```

        </ul>
      )}
    </div>
  );
};

export default MyComponent;

```

■ Axios

```

import React, { useState, useEffect } from 'react';
import axios from 'axios';

const MyComponent = () => {
  const [data, setData] = useState(null);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    const fetchData = async () => {
      try {
        const response = await axios.get('https://api.example.com/data');
        setData(response.data);
        setLoading(false);
      } catch (error) {
        console.error('Error fetching data:', error);
      }
    };

    fetchData();
  }, []);

  if (loading) {
    return <div>Loading...</div>;
  }

  return (
    <div>
      {/* Display fetched data */}
      {data && (
        <ul>
          {data.map(item => (
            <li key={item.id}>{item.name}</li>

```

```
        ))}
      </ul>
    )}
  </div>
);
};

export default MyComponent;
```

- **Performance Optimization**

Performance optimization in React is crucial for creating smooth and responsive user experiences. Here are some key strategies you can employ:

1. Component Optimization

PureComponent/memo: Use `React.PureComponent` or `React.memo` to prevent unnecessary re-renders of components when props haven't changed.

Functional Components: Opt for functional components over class components as they are often simpler and perform better.

Reducer Functions: Break down large components into smaller, more manageable ones to improve rendering and debugging.

Prop Drilling vs. Context: Avoid excessive prop drilling by considering alternatives like `React Context` for sharing data across components.

2. Virtualization

Large Lists: For long lists, use libraries like `react-window` or `react-virtualized` to render only visible items and improve performance.

Infinite Scrolling: Implement efficient infinite scrolling strategies to avoid rendering and loading entire datasets at once.

3. Data Management

Immutable Data: Utilize immutable data structures like `Immutable.js` to avoid unnecessary object mutations and re-renders.

Memoization: Employ memoization techniques to cache expensive calculations and results, particularly for frequently used functions.

4. Rendering optimization

Conditional Rendering: Avoid unnecessary rendering by using conditional statements and short-circuiting operators.

Fragment/React.Fragment: Use React.Fragment to group elements without introducing additional DOM nodes.

5. Code Splitting

Dynamic Imports: Break down your application into smaller bundles using dynamic imports (import() syntax) to reduce initial load time.

Lazy Loading: Load components only when necessary using techniques like lazy loading.

6. Profiling and Monitoring

React DevTools: Use React DevTools to profile component performance and identify bottlenecks.

Performance Libraries: Employ performance monitoring libraries like WebPageTest or Lighthouse to assess overall application performance.

7. Best Practices

- **Minification:** Ensure production builds are minified and compressed for smaller bundle sizes.
- **Network optimization:** Optimize image and asset loading, consider CDNs, and use techniques like HTTP/2 for faster requests.
- **Browser compatibility:** Test and optimize your application for different browsers and devices.

Remember, optimization is an ongoing process. It's essential to identify bottlenecks, prioritize improvements based on impact, and measure the effectiveness of your efforts. By continually applying these strategies and iteratively optimizing, you can ensure your React applications deliver exceptional performance to your users.

- **React Developer Tool**

React DevTools is a browser extension that integrates seamlessly with Chrome DevTools, providing powerful debugging and inspection capabilities for React applications.

- **Installation**

- Install the "React Developer Tools" extension from the Chrome Web Store.
- Open your React application in Chrome.
- Open Chrome DevTools (press F12 or right-click and choose "Inspect").
- Look for the " Components" or " Profiler" tabs next to the Elements, Sources, and Console tabs. These are React DevTools.

- **Components Tab**

- **Component Hierarchy:** View a complete tree structure of your entire React application, showing parent-child relationships between components.
- **Component Details:** Click on any component in the tree to view its name, props, state, context, and lifecycle methods.
- **Prop Examination:** Inspect the values of all props passed to a component, allowing you to understand data flow and component behavior.
- **State Evaluation:** Examine the current state of a component, including individual state variables and nested objects.
- **Context Awareness:** View the context values available to a component and pinpoint potential context-related issues.
- **Lifecycle Method Tracking:** Understand when and how lifecycle methods are invoked for each component.
- **Live Edit:** Modify prop values in real-time and immediately see their impact on the component's rendering and behavior.
- **State Mutation:** Change state values directly in the DevTools to test different scenarios and debug state-related issues.
- **Force Renders:** Force a component to re-render, helping you understand how state changes trigger updates.
- **Search:** Quickly find specific components within the hierarchy using the search bar.
- **Filtering:** Filter components based on type, name, or other criteria for easier navigation.
- **Profiling:** Use the Profiler tab (sometimes nested within Components) to identify performance bottlenecks associated with specific components.
- **Snapshots:** Record component snapshots at different points in time to "time travel" and debug past states.

➤ Profiler Tab

The Profiler Tab in React DevTools dives deeper into your application's performance by providing tools to measure and analyze rendering timelines. Here's a breakdown of its key uses:

- **Start/Stop Recording:** Initiate and stop performance recordings to capture data for specific interactions or sections of your app.
- **Record Frequency:** Adjust the recording frequency to balance detail with performance overhead.
- **Flamegraph View:** Visualize rendering activity as a flamegraph, where wider sections represent more time spent rendering.
- **Identify Bottlenecks:** Locate components and functions that take a significant amount of time to render, highlighting potential performance issues.
- **Component Comparison:** Compare rendering times between different instances of the same component to understand variations.
- **Profile Hook Calls:** Analyze the performance of React Hooks and identify potential optimization opportunities.
- **Identify Expensive Operations:** Pinpoint specific lines of code within components that contribute most to rendering time.
- **Understand Component Interactions:** Visualize how component lifecycles and re-renders interact, informing potential optimizations.
- **Measure Optimization Impact:** Use recordings to quantify the effectiveness of performance improvements you implement.
- **Filter & Search:** Focus on specific components or functions to isolate performance concerns.
- **Export Data:** Export performance data for further analysis in external profiling tools.
- **Custom Profilers:** Develop custom profilers targeting specific aspects of your application's performance.

➤ Beyond Basic Uses

- **Custom Overlays:** Create or use pre-built overlays to visualize specific data or warnings related to your components.
 - **Custom Plugins:** Leverage advanced capabilities by building custom DevTools plugins to tailor the inspection experience to your needs.
 - **Correlate with Other DevTools:** Combine profiling data with insights from the Components Tab and other DevTools features for a holistic understanding of performance.
 - **Learn Performance Optimization Techniques:** Utilize the insights from the Profiler Tab to apply specific optimization strategies like memoization, lazy loading, and code profiling.
-

- **Testing - Refer Testing Documents**
 - **Jest**
 - Tests
 - Matchers
 - Test Runners
 - Test Doubles
 - Test Environments
 - Snapshots
 - Hooks
 - Async and await
 - Timeout and Retry
 - Coverage
 - Custom matchers
 - Parallel testing
 - Watch mode
 - Debugging
 - Custom test runners and reporters
 - Integration with CI/CD
 - **React testing library**
 - User-centric testing
 - Accessibility
 - Render
 - Querying
 - Fireevent
 - Assertions
 - Screen
 - Debug
 - Asynchronous Operations
 - User Events
 - Custom Queries
 - Testing Context
 - Accessibility Testing
 - Testing Router based applications
 - Testing Libraries integration

-
- **Additional references - Vite or react**